Informe Triggers Módulo 9

Configuración y Personalización

Versión: 1.0

25 de abril de 2026

Juan David Peña Roldan

Universidad Surcolombiana

Neiva, Huila

2026

TRG-M9-01 — Unicidad de nombre de especie (case-insensitive)

Descripción	Valida que no exista otra especie con el mismo nombre normalizado (LOWER(TRIM(nombre))). La restricción UNIQUE existente en el esquema es case-sensitive; este trigger cubre la validación case-insensitive exigida por el requerimiento.
Aplicado a	RF-15
Restricción origen RF	"El nombre de la especie debe ser único dentro del catálogo, aplicando normalización case-insensitive (sin distinción entre mayúsculas y minúsculas)." *(RF-15 — Restricciones)*
Justificación	El UNIQUE del DDL no cubre case-insensitive. Sin este trigger, "Bovino" y "bovino" coexistirían.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_especies_nombre_unique_ci()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.especies
    WHERE LOWER(TRIM(nombre)) = LOWER(TRIM(NEW.nombre))
      AND id_especie <> COALESCE(NEW.id_especie, -1);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_SPECIES: Ya existe una especie
con el nombre "%" (validación case-insensitive). Use un nombre
diferente.',
            NEW.nombre
            USING ERRCODE = 'P0101';
    END IF;
```

```
    -- Normalizar nombre: primer carácter en mayúscula, resto en
minúscula
    NEW.nombre := INITCAP(TRIM(NEW.nombre));

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_especies_nombre_unique_ci
BEFORE INSERT OR UPDATE OF nombre ON modulo9.especies
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_especies_nombre_unique_ci();
```

TRG-M9-02 — Longitud y formato mínimo del nombre de especie

Descripción	Valida que el nombre tenga entre 3 y 50 caracteres y no esté vacío tras trim. El constraint CHECK posterior cubre longitud, pero este trigger entrega un mensaje de negocio claro.
Aplicado a	RF-15
Restricción origen RF	"El campo nombre es obligatorio y deberá tener una longitud entre 3 y 50 caracteres." *(RF-15 — Restricciones)*
Justificación	La restricción de longitud mínima de 3 caracteres es un invariante del catálogo que debe protegerse aunque el backend falle.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_especies_nombre_formato()
RETURNS TRIGGER AS $$
DECLARE
    v_longitud INTEGER;
BEGIN
    NEW.nombre := TRIM(NEW.nombre);
    v_longitud := LENGTH(NEW.nombre);

    IF v_longitud < 3 OR v_longitud > 50 THEN
        RAISE EXCEPTION 'INVALID_FORMAT: El nombre de la especie debe
```

```
tener entre 3 y 50 caracteres. Longitud recibida: %.',
            v_longitud
            USING ERRCODE = 'P0102';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_especies_nombre_formato
BEFORE INSERT OR UPDATE OF nombre ON modulo9.especies
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_especies_nombre_formato();
```

TRG-M9-03 — Auditoría automática de operaciones sobre especies

Descripción	Registra en modulo9.gestion_especies cada INSERT o UPDATE sobre especies, capturando el usuario responsable y el timestamp. Cubre el criterio de auditoría obligatoria del RF-15.
Aplicado a	RF-15
Restricción origen RF	"El sistema registra todas las operaciones en el historial de auditoría." "Registro de la operación en el historial de auditoría que incluye: Usuario, Fecha y hora, Tipo de operación (CREATE, UPDATE, DEACTIVATE), Valores anteriores y nuevos." *(RF-15 — Proceso paso 8 y Salida)*
Justificación	La auditoría de cambios sobre el catálogo maestro debe ser automática e incondicional — no puede depender de que el backend recuerde invocarla.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_especies_audit()
RETURNS TRIGGER AS $$
BEGIN
```

```
    INSERT INTO modulo9.gestion_especies (
        id_usuario,
        id_especie,
        fecha_gestion,
        id_umbral_ambiental
    )
    VALUES (
        COALESCE(NEW.id_especie, OLD.id_especie),  -- usa id como proxy
de usuario hasta que el backend provea el id real
        NEW.id_especie,
        now(),
        NULL  -- el umbral se vincula en operaciones posteriores
    );

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_especies_audit
AFTER INSERT OR UPDATE ON modulo9.especies
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_especies_audit();
```

TRG-M9-04 — Protección de eliminación física de especies

Descripción	Bloquea cualquier DELETE físico sobre la tabla especies. Solo se permite la desactivación lógica (es_activo = false).
Aplicado a	RF-15
Restricción origen RF	"No se permite la eliminación física de especies; únicamente se permite su desactivación lógica mediante el campo activo." *(RF-15 — Restricciones)*
Justificación	Es la garantía más básica de la política "sin eliminación física". Un DELETE directo a la BD saltaría el backend.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_especies_no_delete()
```

```
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'NO_PHYSICAL_DELETE: Las especies no pueden
eliminarse físicamente. Use la desactivación lógica (es_activo = false).
Especie afectada: "%".',
        OLD.nombre
        USING ERRCODE = 'P0103';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_especies_no_delete
BEFORE DELETE ON modulo9.especies
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_especies_no_delete();
```

TRG-M9-05 — Unicidad de nombre de ciclo biológico (etapa) por especie, case-insensitive

Descripción	Valida que no exista otra etapa con el mismo nombre normalizado para la misma id_especie. La unicidad es por especie, no global.
Aplicado a	RF-16
Restricción origen RF	"El nombre de cada etapa debe ser obligatorio, único por especie y con longitud entre 3 y 50 caracteres." "Todos los nombres deben validarse como case-insensitive." *(RF-16 — Restricciones)*
Justificación	No existe constraint UNIQUE en el DDL para esta tabla. Sin este trigger, la misma especie podría tener dos etapas llamadas "engorde" y "Engorde".

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_ciclos_biologicos_nombre_unique_ci()
```

```
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.ciclos_biologicos
    WHERE LOWER(TRIM(nombre))  = LOWER(TRIM(NEW.nombre))
      AND id_especie            = NEW.id_especie
      AND id_ciclo_biologico   <> COALESCE(NEW.id_ciclo_biologico, -1);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_STAGE: Ya existe una etapa llamada
"%" para esta especie. Los nombres de etapa deben ser únicos por especie
(case-insensitive).',
            NEW.nombre
            USING ERRCODE = 'P0104';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ciclos_biologicos_nombre_unique_ci
BEFORE INSERT OR UPDATE OF nombre ON modulo9.ciclos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_ciclos_biologicos_nombre_unique_ci();
```

TRG-M9-06 — Validación de duración de ciclo biológico (etapa)

Descripción	Valida que duracion_dias sea un entero positivo estrictamente mayor a 0. El constraint CHECK del DDL no existe para esta tabla.
Aplicado a	RF-16
Restricción origen RF	"La duración de la etapa debe ser un número entero positivo mayor a 0." *(RF-16 — Restricciones)* Flujo alterno: "El usuario ingresa un valor menor o igual a 0 en el campo duracion_dias. HTTP 400: Error de parámetro: La duración estimada de una

	etapa productiva debe ser un número entero positivo mayor a 0 días." *(RF-16 — Flujo alternativo: Duración de etapa inválida)*
Justificación	Invariante numérico estructural. Una duración de 0 o negativa es físicamente incoherente para una etapa productiva.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_ciclos_biologicos_duracion_valida()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.duracion_dias IS NULL OR NEW.duracion_dias <= 0 THEN
        RAISE EXCEPTION 'INVALID_DURATION: La duración de la etapa debe
ser un número entero positivo mayor a 0. Valor recibido: %.',
            NEW.duracion_dias
            USING ERRCODE = 'P0105';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ciclos_biologicos_duracion_valida
BEFORE INSERT OR UPDATE OF duracion_dias ON modulo9.ciclos_biologicos
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_ciclos_biologicos_duracion_valida();
```

TRG-M9-07 — Bloqueo de desactivación de etapa con activos biológicos vinculados

Descripción	Impide desactivar una etapa si existen activos biológicos actualmente en esa fase. Verifica en ciclos_productivos_biologicos si la etapa está referenciada activamente.
Aplicado a	RF-16
Restricción origen RF	"No se puede desactivar una etapa si existen

	activos biológicos en dicha etapa." *(RF-16 — Restricciones)* Flujo alterno: "El administrador intenta marcar como activo: false una etapa que actualmente tiene lotes o animales individuales asociados. HTTP 422: No es posible desactivar la etapa '[NOMBRE_ETAPA]'. Existen [N] activos biológicos actualmente en esta fase del ciclo." *(RF-16 — Flujo alternativo: Desactivación bloqueada por activos biológicos vinculados)*
Justificación	Protección de integridad referencial operativa. Si el backend falla, un UPDATE directo podría desactivar una etapa en uso.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_ciclos_biologicos_no_desactivar_en_uso()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    -- Solo aplica cuando se intenta desactivar (true → false)
    IF OLD.es_activo = TRUE AND NEW.es_activo = FALSE THEN
        SELECT COUNT(*) INTO v_count
        FROM modulo9.ciclos_productivos_biologicos
        WHERE id_ciclo_biologico = OLD.id_ciclo_biologico;

        IF v_count > 0 THEN
            RAISE EXCEPTION 'STAGE_IN_USE: No se puede desactivar la
etapa "%" porque tiene % ciclo(s) productivo(s) vinculado(s). Traslade
los activos antes de proceder.',
                OLD.nombre, v_count
                USING ERRCODE = 'P0106';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ciclos_biologicos_no_desactivar_en_uso
BEFORE UPDATE OF es_activo ON modulo9.ciclos_biologicos
FOR EACH ROW
```

```
EXECUTE FUNCTION
modulo9.trg_fn_ciclos_biologicos_no_desactivar_en_uso();
```

TRG-M9-08 — Unicidad de nombre de patología (case-insensitive, global)

Descripción	Valida unicidad global del nombre de patología de forma case-insensitive. Las patologías no tienen FK directa a especie (la relación va por especies_patologias), por lo que la unicidad es global.
Aplicado a	RF-16
Restricción origen RF	"El nombre de cada patología debe ser obligatorio, único por especie y con longitud entre 3 y 100 caracteres." "Todos los nombres deben validarse como case-insensitive." *(RF-16 — Restricciones)*
Justificación	No existe UNIQUE en el DDL para patologias.nombre. Sin este trigger pueden existir duplicados.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_patologias_nombre_unique_ci()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.patologias
    WHERE LOWER(TRIM(nombre))  = LOWER(TRIM(NEW.nombre))
      AND id_patologias        <> COALESCE(NEW.id_patologias, -1);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_PATHOLOGY: Ya existe una patología
con el nombre "%" en el catálogo global (case-insensitive).',
            NEW.nombre
            USING ERRCODE = 'P0107';
    END IF;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_patologias_nombre_unique_ci
BEFORE INSERT OR UPDATE OF nombre ON modulo9.patologias
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_patologias_nombre_unique_ci();
```

TRG-M9-09 — Protección de eliminación física de patologías

Descripción	Bloquea DELETE físico sobre patologías. Solo se permite desactivación lógica.
Aplicado a	RF-16
Restricción origen RF	"No se permite eliminación física de patologías; solo desactivación lógica." *(RF-16 — Restricciones)*
Justificación	Misma razón que TRG-M9-04 — política "sin eliminación física" de registros del catálogo.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_patologias_no_delete()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'NO_PHYSICAL_DELETE: Las patologías no pueden
eliminarse físicamente. Use la desactivación lógica (es_activo = false).
Patología: "%".',
        OLD.nombre
        USING ERRCODE = 'P0108';
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_patologias_no_delete
BEFORE DELETE ON modulo9.patologias
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_patologias_no_delete();
```

TRG-M9-10 — Unicidad de nombre de métrica productiva (case-insensitive)

Descripción	Valida unicidad global del nombre de métrica productiva de forma case-insensitive. No existe UNIQUE en el DDL para esta tabla.
Aplicado a	RF-16
Restricción origen RF	"El nombre de cada métrica productiva debe ser obligatorio, único por especie y con longitud entre 3 y 50 caracteres." "Todos los nombres deben validarse como case-insensitive." *(RF-16 — Restricciones)*
Justificación	Sin este trigger pueden coexistir "Peso vivo" y "peso vivo" como métricas distintas, rompiendo el RF-16.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_metricas_produccion_nombre_unique_ci()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.metricas_produccion
    WHERE LOWER(TRIM(nombre))      = LOWER(TRIM(NEW.nombre))
      AND id_metrica_produccion   <> COALESCE(NEW.id_metrica_produccion,
-1);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_METRIC: Ya existe una métrica
productiva con el nombre "%" (case-insensitive).',
            NEW.nombre
            USING ERRCODE = 'P0109';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_metricas_produccion_nombre_unique_ci
BEFORE INSERT OR UPDATE OF nombre ON modulo9.metricas_produccion
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_metricas_produccion_nombre_unique_ci();
```

TRG-M9-11 — Unicidad activa de umbral ambiental por especie + variable

Descripción	Impide que existan dos registros activos con la misma combinación `id_especie + id_variable_ambiental`. Si ya existe uno activo, la operación se rechaza — el usuario debe editar el existente o desactivarlo primero.
Aplicado a	RF-17
Restricción origen RF	"No se permitirá la existencia de múltiples configuraciones activas para la misma combinación de especie y variable ambiental." *(RF-17 — Restricciones)* Flujo alterno: "El administrador intenta crear un nuevo registro de 'Temperatura' para la especie 'Bovino' cuando ya existe uno activo. HTTP 409: Ya existe un umbral activo para la variable [TIPO_VARIABLE] en la especie [NOMBRE_ESPECIE]. Por favor, edite la configuración actual en lugar de crear una nueva." *(RF-17 — Flujo alterno: Configuración duplicada para la misma variable)*
Justificación	Restricción de unicidad funcional que el DDL no modela con un UNIQUE parcial (solo sobre registros activos).

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_umbral_ambiental_duplicado()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
```

```
BEGIN
    -- Solo validar si el registro nuevo/actualizado está activo
    IF NEW.es_activo = TRUE THEN
        SELECT COUNT(*) INTO v_count
        FROM modulo9.umbrales_ambientales
        WHERE id_especie          = NEW.id_especie
          AND id_variable_ambiental = NEW.id_variable_ambiental
          AND es_activo           = TRUE
          AND id_umbral_ambiental <> COALESCE(NEW.id_umbral_ambiental,
-1);

        IF v_count > 0 THEN
            RAISE EXCEPTION 'DUPLICATE_THRESHOLD: Ya existe un umbral
activo para esta combinación de especie y variable ambiental. Edite el
umbral existente en lugar de crear uno nuevo.'
                USING ERRCODE = 'P0110';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_umbral_ambiental_duplicado
BEFORE INSERT OR UPDATE ON modulo9.umbrales_ambientales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_umbral_ambiental_duplicado();
```

TRG-M9-12 — Validación de rango y especie activa en umbral ambiental

Descripción	Valida dos invariantes: (1) valor_min < valor_max, y (2) la especie referenciada está activa. Si la especie está inactiva, no se puede configurar un umbral sobre ella.
Aplicado a	RF-17
Restricción origen RF	"Los valores mínimos definidos para cada variable ambiental deben ser estrictamente menores que los valores máximos." "Los umbrales ambientales solo podrán configurarse para especies que se

	encuentren activas dentro del catálogo del sistema." *(RF-17 — Restricciones)* Flujo alterno: "El usuario intenta guardar una configuración donde el valor_min es igual o superior al valor_max. HTTP 400: El valor mínimo ([VALOR_MIN]) no puede ser mayor o igual al valor máximo ([VALOR_MAX])." *(RF-17 — Flujo alterno: Inconsistencia de rango)* Flujo alterno: "Se intenta asociar un umbral a un id_especie que no existe o que fue marcado como activo: false. HTTP 422: No se pueden configurar umbrales para la especie seleccionada porque se encuentra inactiva o no existe." *(RF-17 — Flujo alterno: Especie inactiva o no encontrada)*
Justificación	Son invariantes matemáticos y relacionales básicos. Un umbral con min >= max es incoherente estructuralmente.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_umbral_ambiental_rango_valido()
RETURNS TRIGGER AS $$
DECLARE
    v_especie_activa BOOLEAN;
BEGIN
    -- Validar rango lógico
    IF NEW.valor_min >= NEW.valor_max THEN
        RAISE EXCEPTION 'INVALID_RANGE: El valor mínimo (%) no puede ser
mayor o igual al valor máximo (%) para el umbral ambiental.',
            NEW.valor_min, NEW.valor_max
            USING ERRCODE = 'P0111';
    END IF;

    -- Validar que la especie esté activa
    SELECT es_activo INTO v_especie_activa
    FROM modulo9.especies
    WHERE id_especie = NEW.id_especie;

    IF v_especie_activa IS NULL OR v_especie_activa = FALSE THEN
        RAISE EXCEPTION 'INACTIVE_SPECIES: No se puede configurar un
umbral para la especie ID % porque está inactiva o no existe.',
            NEW.id_especie
```

```
            USING ERRCODE = 'P0112';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_umbral_ambiental_rango_valido
BEFORE INSERT OR UPDATE ON modulo9.umbrales_ambientales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_umbral_ambiental_rango_valido();
```

TRG-M9-13 — Validación de no solapamiento de niveles de alerta

Descripción	Verifica que el rango [limite_inferior, limite_superior] del nuevo nivel no se solape con ningún rango ya registrado para el mismo id_umbral_ambiental. Además valida que limite_inferior < limite_superior.
Aplicado a	RF-17
Restricción origen RF	"Los rangos definidos para los niveles de alerta no deben solaparse entre sí y deben mantener coherencia lógica dentro del rango general definido." > "El sistema no permitirá guardar configuraciones donde los valores definidos generen inconsistencias en los rangos de monitoreo o en los niveles de alerta." *(RF-17 — Restricciones)* > Flujo alterno: "El usuario ingresa valores que se cruzan entre sí. HTTP 400: Los niveles de alerta presentan solapamientos. El límite superior del Nivel de Precaución debe ser exactamente igual al límite inferior del Nivel Crítico para garantizar una transición lineal." *(RF-17 — Flujo alterno: Solapamiento de niveles de alerta)*
Justificación	Invariante matemático sobre rangos. Dos rangos solapados producen ambigüedad en

	la clasificación de alertas — es un error estructural, no de lógica de negocio.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_niveles_alerta_solapamiento()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    -- Validar coherencia interna del nivel
    IF NEW.limite_inferior >= NEW.limite_superior THEN
        RAISE EXCEPTION 'INVALID_ALERT_RANGE: El límite inferior (%)
debe ser estrictamente menor al límite superior (%) del nivel de
alerta.',
            NEW.limite_inferior, NEW.limite_superior
            USING ERRCODE = 'P0113';
    END IF;

    -- Detectar solapamiento con otros niveles del mismo umbral
    SELECT COUNT(*) INTO v_count
    FROM modulo9.niveles_alerta_ambientales
    WHERE id_umbral_ambiental         = NEW.id_umbral_ambiental
      AND id_nivel_alerta_ambiental  <>
COALESCE(NEW.id_nivel_alerta_ambiental, -1)
      AND (
            NEW.limite_inferior < limite_superior
            AND NEW.limite_superior > limite_inferior
          );

    IF v_count > 0 THEN
        RAISE EXCEPTION 'OVERLAPPING_ALERT: El rango [%, %] del nuevo
nivel de alerta se solapa con un rango ya existente para este umbral
ambiental.',
            NEW.limite_inferior, NEW.limite_superior
            USING ERRCODE = 'P0114';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_niveles_alerta_solapamiento
BEFORE INSERT OR UPDATE ON modulo9.niveles_alerta_ambientales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_niveles_alerta_solapamiento();
```

TRG-M9-14 — Unicidad de configuración global activa

Descripción	Garantiza que solo exista un registro con es_activo = true en la tabla de configuraciones globales. Si ya existe uno activo, bloquea el INSERT.
Aplicado a	RF-18
Restricción origen RF	"Solo podrá existir una configuración activa de parámetros operativos en el sistema." > "No se permite la existencia de múltiples configuraciones activas simultáneamente." *(RF-18 — Restricciones)* > Flujo alterno: "El sistema detecta un intento de insertar un nuevo registro (POST) en lugar de actualizar el existente (PUT/PATCH). HTTP 409: Ya existe una configuración operativa activa. Utilice el método de actualización para modificar los valores vigentes." *(RF-18 — Flujo alterno: Intento de creación de múltiples configuraciones activas)*
Justificación	Invariante de unicidad global que el DDL no modela con un UNIQUE parcial. Sin este trigger pueden coexistir múltiples configuraciones activas.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_configuracion_global_unicidad()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    IF NEW.es_activo = TRUE THEN
        SELECT COUNT(*) INTO v_count
        FROM modulo9.configuraciones_globales
        WHERE es_activo = TRUE;

        IF v_count > 0 THEN
            RAISE EXCEPTION 'DUPLICATE_GLOBAL_CONFIG: Ya existe una
configuración global activa. Actualice la configuración vigente en lugar
```

```
de crear una nueva.'
                USING ERRCODE = 'P0115';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_configuracion_global_unicidad
BEFORE INSERT ON modulo9.configuraciones_globales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_configuracion_global_unicidad();
```

TRG-M9-15 — Validación de heartbeat y frecuencia de muestreo

Descripción	Valida que `frecuencia_muestreo > 0`, `heartbeat > 0`, y que `heartbeat >= frecuencia_muestreo`. Son los tres invariantes del RF-18.
Aplicado a	RF-18
Restricción origen RF	"Los valores deben ser números enteros positivos expresados en minutos." "El valor de heartbeat debe ser mayor o igual que la frecuencia de muestreo configurada." *(RF-18 — Restricciones)* Flujo alterno: "El Administrador ingresa un valor ≤ 0 en frecuencia_muestreo o heartbeat. HTTP 400: Los parámetros operativos deben ser números enteros positivos mayores a cero." *(RF-18 — Flujo alterno: Valores no permitidos)* > Flujo alterno: "El usuario intenta configurar un heartbeat menor a la frecuencia_muestreo. HTTP 400: El tiempo de espera (heartbeat) debe ser mayor o igual a la frecuencia de muestreo." *(RF-18 — Flujo alterno: Inconsistencia

	lógica entre Muestreo y Heartbeat)*
Justificación	Reglas matemáticas básicas de coherencia operativa. Un heartbeat menor que la frecuencia de muestreo es físicamente imposible de cumplir.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_configuracion_global_heartbeat_valido()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.frecuencia_muestreo IS NULL OR NEW.frecuencia_muestreo <= 0
THEN
        RAISE EXCEPTION 'INVALID_CONFIG: La frecuencia de muestreo debe
ser un entero positivo mayor a 0. Valor recibido: %.',
            NEW.frecuencia_muestreo
            USING ERRCODE = 'P0116';
    END IF;

    IF NEW.heartbeat IS NULL OR NEW.heartbeat <= 0 THEN
        RAISE EXCEPTION 'INVALID_CONFIG: El heartbeat debe ser un entero
positivo mayor a 0. Valor recibido: %.',
            NEW.heartbeat
            USING ERRCODE = 'P0116';
    END IF;

    IF NEW.heartbeat < NEW.frecuencia_muestreo THEN
        RAISE EXCEPTION 'INVALID_CONFIG: El heartbeat (%) debe ser mayor
o igual a la frecuencia de muestreo (%). No puede esperarse una señal
antes del intervalo de envío.',
            NEW.heartbeat, NEW.frecuencia_muestreo
            USING ERRCODE = 'P0117';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_configuracion_global_heartbeat_valido
BEFORE INSERT OR UPDATE ON modulo9.configuraciones_globales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_configuracion_global_heartbeat_valido();
```

TRG-M9-16 — Unicidad y validación de formato del nombre de finca

Descripción	Valida unicidad del nombre de finca tanto a nivel global como por productor (id_usuario). También valida que el nombre solo contenga letras, espacios y caracteres del español (vocales acentuadas, ñ).
Aplicado a	RF-19
Restricción origen RF	"El nombre de la finca debe ser único (de manera global y por productor) dentro del sistema." "El nombre de la finca y los atributos de ubicacion_finca deben permitir letras del alfabeto (A–Z, a–z), espacios y caracteres propios del idioma español, tales como vocales acentuadas (á, é, í, ó, ú) y la letra ñ. No se permiten números ni caracteres especiales (símbolos) distintos a los mencionados." *(RF-19 — Restricciones)* Flujo alterno: "El sistema detecta que la combinación de nombre_finca ya existen para el mismo productor_id o a nivel global. HTTP 409: Ya existe una finca con el nombre '[NOMBRE_INGRESADO]' en la ubicación seleccionada." *(RF-19 — Flujo alterno: Nombre o ubicación de finca duplicados)* Flujo alterno: "Se detectan números o símbolos especiales en nombre_finca, departamento, municipio o vereda. HTTP 400: Los campos de nombre y ubicación solo permiten letras y espacios." *(RF-19 — Flujo alterno: Caracteres no permitidos en campos de texto)*
Justificación	No existe UNIQUE en el DDL para fincas.nombre. Sin este trigger pueden registrarse fincas duplicadas para el mismo productor.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_finca_nombre_unique()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_count_global   INTEGER;
    v_count_productor INTEGER;
BEGIN
    NEW.nombre := TRIM(NEW.nombre);

    -- Validar formato: solo letras, espacios, acentos y ñ
    IF NEW.nombre !~ '^[A-Za-záéíóúÁÉÍÓÚñÑüÜ\s]+$' THEN
        RAISE EXCEPTION 'INVALID_FORMAT: El nombre de la finca solo
permite letras, espacios y caracteres del español. No se admiten números
ni símbolos. Valor: "%".',
            NEW.nombre
            USING ERRCODE = 'P0118';
    END IF;

    -- Validar unicidad global
    SELECT COUNT(*) INTO v_count_global
    FROM modulo9.fincas
    WHERE LOWER(TRIM(nombre)) = LOWER(NEW.nombre)
      AND id_finca            <> COALESCE(NEW.id_finca, -1);

    IF v_count_global > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_FARM_GLOBAL: Ya existe una finca con
el nombre "%" en el sistema (unicidad global).',
            NEW.nombre
            USING ERRCODE = 'P0119';
    END IF;

    -- Validar unicidad por productor
    IF NEW.id_usuario IS NOT NULL THEN
        SELECT COUNT(*) INTO v_count_productor
        FROM modulo9.fincas
        WHERE LOWER(TRIM(nombre)) = LOWER(NEW.nombre)
          AND id_usuario          = NEW.id_usuario
          AND id_finca           <> COALESCE(NEW.id_finca, -1);

        IF v_count_productor > 0 THEN
            RAISE EXCEPTION 'DUPLICATE_FARM_PRODUCER: El productor ya
tiene una finca registrada con el nombre "%".',
                NEW.nombre
                USING ERRCODE = 'P0120';
        END IF;
    END IF;

    RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_finca_nombre_unique
BEFORE INSERT OR UPDATE OF nombre ON modulo9.fincas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_finca_nombre_unique();
```

TRG-M9-17 — Validación de coordenadas y tamaño de finca

Descripción	Extrae latitud y longitud del campo JSONB ubicacion y valida sus rangos. También valida que tamano_h > 0.
Aplicado a	RF-19
Restricción origen RF	"latitud se debe encontrar dentro del rango permitido (-90 a 90)." "longitud se debe encontrar dentro del rango permitido (-180 a 180)." "El tamaño de la finca debe ser mayor a cero y puede manejar decimales." *(RF-19 — Restricciones)* Flujo alterno: "El usuario ingresa una latitud fuera del rango (-90 a 90) o una longitud fuera de (-180 a 180). HTTP 400: Las coordenadas ingresadas son inválidas." *(RF-19 — Flujo alterno: Coordenadas geográficas fuera de rango)* Flujo alterno: "El Administrador ingresa un valor ≤ 0 en el campo tamaño_h. HTTP 400: El tamaño de la finca debe ser un valor numérico mayor a cero hectáreas." *(RF-19 — Flujo alterno: Tamaño de finca inválido)*
Justificación	Son invariantes numéricos básicos. Coordenadas fuera de rango son físicamente imposibles; el campo JSONB no puede protegerlos con un CHECK simple.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_finca_coordenadas_validas()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_lat  NUMERIC;
    v_lng  NUMERIC;
BEGIN
    -- Validar tamaño
    IF NEW.tamano_h IS NULL OR NEW.tamano_h <= 0 THEN
        RAISE EXCEPTION 'INVALID_SIZE: El tamaño de la finca debe ser un
valor numérico positivo mayor a cero. Valor recibido: %.',
            NEW.tamano_h
            USING ERRCODE = 'P0121';
    END IF;

    -- Extraer y validar coordenadas del JSONB
    IF NEW.ubicacion IS NOT NULL THEN
        v_lat := (NEW.ubicacion->>'latitud')::NUMERIC;
        v_lng := (NEW.ubicacion->>'longitud')::NUMERIC;

        IF v_lat IS NOT NULL AND (v_lat < -90 OR v_lat > 90) THEN
            RAISE EXCEPTION 'INVALID_COORDINATES: La latitud debe estar
entre -90 y 90. Valor recibido: %.',
                v_lat
                USING ERRCODE = 'P0122';
        END IF;

        IF v_lng IS NOT NULL AND (v_lng < -180 OR v_lng > 180) THEN
            RAISE EXCEPTION 'INVALID_COORDINATES: La longitud debe estar
entre -180 y 180. Valor recibido: %.',
                v_lng
                USING ERRCODE = 'P0122';
        END IF;
    END IF;

    RETURN NEW;

EXCEPTION
    WHEN invalid_text_representation THEN
        RAISE EXCEPTION 'INVALID_JSON_COORDINATES: Las coordenadas en el
campo ubicacion no tienen formato numérico válido.'
            USING ERRCODE = 'P0123';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_finca_coordenadas_validas
BEFORE INSERT OR UPDATE ON modulo9.fincas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_finca_coordenadas_validas();
```

TRG-M9-18 — Protección de eliminación física de fincas

Descripción	Bloquea DELETE físico sobre fincas con registros asociados (infraestructuras, dispositivos IoT, activos biológicos). Solo se permite desactivación lógica.
Aplicado a	RF-19
Restricción origen RF	"No se permite eliminar una finca que tenga registros asociados en el sistema; únicamente se permite cambiar su estado a inactiva." *(RF-19 — Restricciones)* Flujo alterno: "El Administrador intenta marcar una finca como activo: false, pero existen dispositivos IoT (RF-21) o Activos Biológicos (RF-33) vinculados a ella. HTTP 422: No se puede desactivar la finca '[NOMBRE_FINCA]' porque aún cuenta con infraestructura o activos biológicos asociados." *(RF-19 — Flujo alterno: Intento de desactivación con dependencias activas)*
Justificación	La FK con ON DELETE NO ACTION generaría un error genérico. Este trigger entrega un mensaje de negocio controlado y verificado.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_finca_no_delete_con_dependencias()
RETURNS TRIGGER AS $$
DECLARE
    v_infra_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_infra_count
    FROM modulo9.infraestructuras
    WHERE id_finca = OLD.id_finca;

    IF v_infra_count > 0 THEN
        RAISE EXCEPTION 'FARM_HAS_DEPENDENCIES: La finca "%" tiene %
infraestructura(s) asociada(s) y no puede eliminarse físicamente. Use la
desactivación lógica (es_activo = false).',
```

```
            OLD.nombre, v_infra_count
            USING ERRCODE = 'P0124';
    END IF;

    RAISE EXCEPTION 'NO_PHYSICAL_DELETE: Las fincas no pueden eliminarse
físicamente del sistema. Use la desactivación lógica (es_activo =
false). Finca: "%".',
        OLD.nombre
        USING ERRCODE = 'P0124';

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_finca_no_delete
BEFORE DELETE ON modulo9.fincas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_finca_no_delete_con_dependencias();
```

TRG-M9-19 — Unicidad de nombre de infraestructura por finca (case-insensitive)

Descripción	Complementa el UNIQUE (nombre, id_finca) del DDL con validación case-insensitive. El constraint existente es case-sensitive.
Aplicado a	RF-20
Restricción origen RF	"El nombre del área productiva debe ser único dentro de la misma finca, evitando duplicidad de áreas dentro de una misma área productiva." *(RF-20 — Restricciones)* Flujo alterno: "El sistema detecta que el nombre_infraestructura ya existe para el finca_id proporcionado (validación case-insensitive). HTTP 409: Ya existe un área denominada '[NOMBRE_INGRESADO]' en esta finca." *(RF-20 — Flujo alterno: Nombre de área

	productiva duplicado en la misma finca)*
Justificación	"Galpon Norte" y "galpon norte" pasarían el constraint del DDL pero son el mismo nombre — el trigger los bloquea.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_infraestructura_nombre_unique_ci()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.infraestructuras
    WHERE LOWER(TRIM(nombre))    = LOWER(TRIM(NEW.nombre))
      AND id_finca               = NEW.id_finca
      AND id_infraestructura    <> COALESCE(NEW.id_infraestructura, -1);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_AREA: Ya existe un área productiva
con el nombre "%" en esta finca (case-insensitive).',
            NEW.nombre
            USING ERRCODE = 'P0125';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_infraestructura_nombre_unique_ci
BEFORE INSERT OR UPDATE OF nombre ON modulo9.infraestructuras
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_infraestructura_nombre_unique_ci();
```

TRG-M9-20 — Validación de superficie de infraestructura

Descripción	Valida que superficie > 0. El constraint CHECK del DDL no existe para esta tabla.
Aplicado a	RF-20

Restricción origen RF	"La superficie del área productiva debe ser un valor numérico positivo, representando el tamaño físico del área (Metros cuadrados)." *(RF-20 — Restricciones)* Flujo alterno: "El Administrador ingresa un valor ≤ 0 en el campo superficie. HTTP 400: La superficie del área productiva debe ser un valor numérico positivo." *(RF-20 — Flujo alterno: Superficie inválida)*
Justificación	Invariante numérico estructural. Una superficie de 0 o negativa es físicamente imposible.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_infraestructura_superficie_valida()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.superficie IS NULL OR NEW.superficie <= 0 THEN
        RAISE EXCEPTION 'INVALID_SURFACE: La superficie del área
productiva debe ser un valor positivo mayor a cero. Valor recibido: %.',
            NEW.superficie
            USING ERRCODE = 'P0126';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_infraestructura_superficie_valida
BEFORE INSERT OR UPDATE OF superficie ON modulo9.infraestructuras
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_infraestructura_superficie_valida();
```

TRG-M9-21 — Bloqueo de desactivación de infraestructura con dependencias operativas

Descripción	Impide desactivar una infraestructura si

	tiene dispositivos IoT activos o sensores actualmente asociados.
Aplicado a	RF-20
Restricción origen RF	"No se permite eliminar áreas productivas que tengan registros asociados (como dispositivos IoT o procesos productivos); únicamente podrán marcarse como inactivas." *(RF-20 — Restricciones)* Flujo alterno: "El Administrador intenta marcar como activo: false un área que tiene sensores IoT vinculados o activos biológicos actualmente alojados en ella. HTTP 422: El área '[NOMBRE_AREA]' tiene [N] dispositivos y/o [M] activos biológicos asociados. Debe desvincular o trasladar los recursos antes de desactivar la infraestructura." *(RF-20 — Flujo alterno: Desactivación bloqueada por dependencias operativas)*
Justificación	Protección de integridad referencial operativa. Sin este trigger un UPDATE directo podría desactivar un área con sensores activos.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_infraestructura_no_desactivar_en_uso()
RETURNS TRIGGER AS $$
DECLARE
    v_dispositivos INTEGER;
    v_sensores     INTEGER;
BEGIN
    IF OLD.es_activo = TRUE AND NEW.es_activo = FALSE THEN

        SELECT COUNT(*) INTO v_dispositivos
        FROM modulo9.dispositivos_iot
        WHERE id_infraestructura = OLD.id_infraestructura
          AND es_activo = TRUE;

        SELECT COUNT(*) INTO v_sensores
        FROM modulo9.sensores_areas_asociadas
        WHERE id_infraestructura = OLD.id_infraestructura
          AND tiene_estado = TRUE;
```

```
        IF v_dispositivos > 0 OR v_sensores > 0 THEN
            RAISE EXCEPTION 'AREA_IN_USE: El área "%" tiene %
dispositivo(s) activo(s) y % sensor(es) asociado(s). Desvincule los
recursos antes de desactivar la infraestructura.',
                OLD.nombre, v_dispositivos, v_sensores
                USING ERRCODE = 'P0127';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_infraestructura_no_desactivar_en_uso
BEFORE UPDATE OF es_activo ON modulo9.infraestructuras
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_infraestructura_no_desactivar_en_uso();
```

TRG-M9-22 — Unicidad del serial de dispositivo IoT

Descripción	Verifica que el serial del dispositivo no esté previamente registrado. El UNIQUE constraint no existe en el DDL actual para esta columna.
Aplicado a	RF-21
Restricción origen RF	"El serial del dispositivo no puede repetirse dentro del sistema." *(RF-21 — Restricciones)* Flujo alterno: "El usuario intenta registrar un dispositivo con un serial que ya existe en la base de datos global. HTTP 409: El número de serie '[SERIAL_INGRESADO]' ya se encuentra vinculado al dispositivo [ID_INTERNO]. Cada equipo físico debe tener un registro único." *(RF-21 — Flujo alterno: Serial físico duplicado)*
Justificación	El serial es el identificador físico del hardware. Dos registros con el mismo serial representarían el mismo equipo físico — es

	un invariante de identidad.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_dispositivo_serial_unique()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.dispositivos_iot
    WHERE TRIM(serial) = TRIM(NEW.serial);

    IF v_count > 0 THEN
        RAISE EXCEPTION 'DUPLICATE_SERIAL: El número de serie "%" ya
está registrado en el sistema. Cada dispositivo físico debe tener un
único registro.',
            NEW.serial
            USING ERRCODE = 'P0128';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_dispositivo_serial_unique
BEFORE INSERT ON modulo9.dispositivos_iot
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_dispositivo_serial_unique();
```

TRG-M9-23 — Bloqueo de eliminación física de dispositivos IoT con datos históricos

Descripción	Impide el DELETE físico de un dispositivo que tiene calibraciones, configuraciones remotas o sensores asociados.
Aplicado a	RF-21
Restricción origen RF	"No se permite eliminar dispositivos con datos históricos asociados; únicamente

	pueden marcarse como inactivos." *(RF-21 — Restricciones)* Flujo alterno: "El usuario intenta ejecutar un método DELETE sobre un dispositivo que ya tiene registros de telemetría o alertas asociadas. HTTP 405: El dispositivo [SERIAL] tiene datos históricos vinculados y no puede ser eliminado. Utilice la opción de desactivación lógica (activo: false)." *(RF-21 — Flujo alterno: Intento de eliminación física)*
Justificación	La FK con ON DELETE NO ACTION generaría error genérico. Este trigger entrega un mensaje controlado y verificado.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_dispositivo_no_delete_con_historial()
RETURNS TRIGGER AS $$
DECLARE
    v_calibraciones  INTEGER;
    v_configuraciones INTEGER;
    v_sensores       INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_calibraciones
    FROM modulo9.calibraciones
    WHERE id_dispositivo_iot = OLD.id_dispositivo_iot;

    SELECT COUNT(*) INTO v_configuraciones
    FROM modulo9.configuraciones_remotas
    WHERE id_dispositivo_iot = OLD.id_dispositivo_iot;

    SELECT COUNT(*) INTO v_sensores
    FROM modulo9.sensores
    WHERE id_dispositivo_iot = OLD.id_dispositivo_iot;

    IF v_calibraciones > 0 OR v_configuraciones > 0 OR v_sensores > 0
THEN
        RAISE EXCEPTION 'DEVICE_HAS_HISTORY: El dispositivo con serial
"%" tiene datos históricos asociados (% calibración(es), %
configuración(es), % sensor(es)). Use la desactivación lógica (es_activo
= false).',
            OLD.serial, v_calibraciones, v_configuraciones, v_sensores
            USING ERRCODE = 'P0129';
    END IF;
```

```
    RAISE EXCEPTION 'NO_PHYSICAL_DELETE: Los dispositivos IoT no pueden
eliminarse físicamente. Use la desactivación lógica (es_activo = false).
Serial: "%".',
        OLD.serial
        USING ERRCODE = 'P0129';

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_dispositivo_no_delete
BEFORE DELETE ON modulo9.dispositivos_iot
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_dispositivo_no_delete_con_historial();
```

TRG-M9-24 — Unicidad de asociación activa sensor-área

Descripción	Verifica que el sensor no tenga ya una asociación activa (tiene_estado = true) con otra área. Un sensor solo puede monitorearse en una área a la vez.
Aplicado a	RF-22
Restricción origen RF	"Un sensor solo puede estar asociado a una única área productiva activa a la vez." "No se permite registrar asociaciones duplicadas entre el mismo sensor y la misma área productiva." *(RF-22 — Restricciones)* Flujo alterno: "El sistema detecta que el sensor ya tiene una relación activa con otra área en la tabla de asociaciones. HTTP 409: El sensor [ID_SENSOR] ya está monitoreando el área '[NOMBRE_AREA_ACTUAL]'. ¿Desea reasignarlo? Esta acción finalizará la asociación anterior automáticamente." *(RF-22 — Flujo alterno: Sensor ya asociado a otra área)*
Justificación	Invariante de cardinalidad 1:1 activa entre sensor y área. Sin este trigger un sensor podría estar activo en dos lugares

	simultáneamente.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_sensor_asociacion_unica_activa()
RETURNS TRIGGER AS $$
DECLARE
    v_count      INTEGER;
    v_nombre_area VARCHAR(50);
BEGIN
    SELECT COUNT(*), i.nombre
    INTO v_count, v_nombre_area
    FROM modulo9.sensores_areas_asociadas saa
    JOIN modulo9.infraestructuras i ON i.id_infraestructura =
saa.id_infraestructura
    WHERE saa.id_sensor    = NEW.id_sensor
      AND saa.tiene_estado = TRUE
    GROUP BY i.nombre
    LIMIT 1;

    IF v_count > 0 THEN
        RAISE EXCEPTION 'SENSOR_ALREADY_ASSIGNED: El sensor ID % ya está
activo en el área "%". Finalice la asociación actual antes de
reasignarlo.',
            NEW.id_sensor, v_nombre_area
            USING ERRCODE = 'P0130';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sensor_asociacion_unica_activa
BEFORE INSERT ON modulo9.sensores_areas_asociadas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_sensor_asociacion_unica_activa();
```

TRG-M9-25 — Validación de dispositivo activo para calibración

Descripción	Verifica que el dispositivo IoT al que se asocia la calibración esté activo (es_activo =

	true). No se pueden calibrar dispositivos inactivos.
Aplicado a	RF-24
Restricción origen RF	"Solo se podrán calibrar sensores pertenecientes a dispositivos activos." *(RF-24 — Restricciones)* Flujo alterno: "El dispositivo_iot_id se encuentra en estado activo: false. HTTP 422: El dispositivo [SERIAL] está inactivo. Debe activar el dispositivo antes de proceder con el registro de nuevos parámetros de calibración." *(RF-24 — Flujo alterno: Dispositivo inactivo)*
Justificación	Restricción de integridad operativa. Una calibración sobre un dispositivo inactivo es incoherente y no debe persistirse.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_calibracion_dispositivo_activo()
RETURNS TRIGGER AS $$
DECLARE
    v_es_activo BOOLEAN;
    v_serial    VARCHAR(50);
BEGIN
    SELECT es_activo, serial
    INTO v_es_activo, v_serial
    FROM modulo9.dispositivos_iot
    WHERE id_dispositivo_iot = NEW.id_dispositivo_iot;

    IF v_es_activo IS NULL THEN
        RAISE EXCEPTION 'DEVICE_NOT_FOUND: El dispositivo IoT con ID %
no existe en el sistema.',
            NEW.id_dispositivo_iot
            USING ERRCODE = 'P0131';
    END IF;

    IF v_es_activo = FALSE THEN
        RAISE EXCEPTION 'INACTIVE_DEVICE: No se puede registrar una
calibración para el dispositivo "%" porque está inactivo. Active el
dispositivo antes de calibrar.',
            v_serial
            USING ERRCODE = 'P0132';
    END IF;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_calibracion_dispositivo_activo
BEFORE INSERT ON modulo9.calibraciones
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_calibracion_dispositivo_activo();
```

TRG-M9-26 — Unicidad de nombre de plantilla e inmutabilidad de versiones

Descripción	Valida unicidad global del `template_name`. Si el nombre ya existe, asigna automáticamente la siguiente versión incremental (`MAX(version) + 1`), implementando la regla de inmutabilidad — no se sobrescriben versiones, se crean nuevas.
Aplicado a	RF-30, RF-31
Restricción origen RF	"El nombre de la plantilla debe ser único en el sistema." "Cada plantilla almacena un número de versión entero incremental. Una plantilla guardada no puede modificarse; cualquier actualización genera una nueva versión." *(RF-30 — Restricciones)* "El sistema rechaza la creación de una plantilla con un nombre ya existente." "Un intento de modificar una plantilla existente genera una nueva versión, no sobreescribe la original." *(RF-31 — Criterios de aceptación)* Flujo alterno: "El sistema detecta que el template_name ya existe en el registro global de plantillas. HTTP 409: Ya existe una plantilla denominada '[NOMBRE_INGRESADO]'. Por favor, asigne un nombre único o genere una nueva versión." *(RF-30 / RF-31 — Flujo alterno: Nombre

	de plantilla duplicado)*
Justificación	La inmutabilidad de las plantillas es un invariante estructural crítico. Si el backend intenta sobrescribir, el trigger lo convierte en una nueva versión.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_plantilla_version_incremental()
RETURNS TRIGGER AS $$
DECLARE
    v_max_version INTEGER;
BEGIN
    -- Verificar si ya existe una plantilla con ese nombre
    SELECT MAX(version) INTO v_max_version
    FROM modulo9.plantillas
    WHERE LOWER(TRIM(template_name)) = LOWER(TRIM(NEW.template_name));

    IF v_max_version IS NULL THEN
        -- Primera versión de esta plantilla
        NEW.version := 1;
    ELSE
        -- Versión incremental -- la plantilla anterior no se modifica
        NEW.version := v_max_version + 1;
    END IF;

    NEW.fecha_creacion := now();

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_plantilla_version_incremental
BEFORE INSERT ON modulo9.plantillas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_plantilla_version_incremental();
```

TRG-M9-27 — Inmutabilidad de plantillas (bloqueo de UPDATE y DELETE)

Descripción	Bloquea cualquier UPDATE o DELETE sobre una plantilla ya creada. Las plantillas son inmutables — cualquier cambio debe generar una nueva versión vía INSERT.
Aplicado a	RF-30, RF-31
Restricción origen RF	"Una plantilla creada no puede modificarse directamente; cualquier actualización genera una nueva versión con número incremental." *(RF-31 — Restricciones)* "Las plantillas son inmutables. Para realizar cambios, debe generar una nueva versión." *(RF-30 — Flujo alterno: Intento de modificación de plantilla existente)* Flujo alterno: "El usuario intenta realizar una petición de creación (POST) sobre una combinación que ya existe. HTTP 405: Las versiones de las plantillas son inmutables." *(RF-31 — Flujo alterno: Intento de sobrescritura)*
Justificación	Invariante de inmutabilidad. Sin este trigger un UPDATE directo modificaría una versión existente sin rastro.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_plantilla_inmutable()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION 'IMMUTABLE_TEMPLATE: La plantilla "%" (v%) es
inmutable y no puede eliminarse. Las plantillas son registros
permanentes.',
            OLD.template_name, OLD.version
            USING ERRCODE = 'P0134';
    END IF;

    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION 'IMMUTABLE_TEMPLATE: La plantilla "%" (v%) es
inmutable. Para actualizar, genere una nueva versión mediante INSERT.',
            OLD.template_name, OLD.version
            USING ERRCODE = 'P0134';
    END IF;

    RETURN NULL;
END;
```

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_plantilla_inmutable_update
BEFORE UPDATE ON modulo9.plantillas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_plantilla_inmutable();

CREATE TRIGGER trg_plantilla_inmutable_delete
BEFORE DELETE ON modulo9.plantillas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_plantilla_inmutable();
```

TRG-M9-28 — Auditoría de cambios en identidad visual

Descripción	Registra en modulo9.auditorias_visuales el estado anterior y nuevo de cada modificación, con usuario y timestamp.
Aplicado a	RF-26
Restricción origen RF	"El sistema registrará la modificación en el historial de auditoría, incluyendo usuario, fecha, hora y valores anteriores y nuevos de cada elemento modificado." *(RF-26 — Proceso paso 10)* "Los cambios realizados deben mantenerse consistentes en todas las vistas de la plataforma." *(RF-26 — Restricciones)*
Justificación	La auditoría de identidad visual debe ser automática e incondicional. El RF-26 exige trazabilidad completa de todos los cambios.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_identidad_visual_audit()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO modulo9.auditorias_visuales (
        id_usuario,
        fecha_creacion,
        valor_anterior,
        valor_nuevo
```

```
    )
    VALUES (
        NEW.id_usuario,
        now(),
        jsonb_build_object(
            'logo_path',        OLD.logo_path,
            'primary_color',    OLD.primary_color,
            'secondary_color',  OLD.secondary_color,
            'org_display_name',OLD.org_display_name,
            'version',          OLD.version
        ),
        jsonb_build_object(
            'logo_path',        NEW.logo_path,
            'primary_color',    NEW.primary_color,
            'secondary_color',  NEW.secondary_color,
            'org_display_name',NEW.org_display_name,
            'version',          NEW.version
        )
    );

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_identidad_visual_audit
AFTER UPDATE ON modulo9.identidad_visuales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_identidad_visual_audit();
```

TRG-M9-29 — Validación de colores hexadecimales en identidad visual

Descripción	Valida que `primary_color` y `secondary_color` cumplan el formato hexadecimal de 6 dígitos (`#RRGGBB`).
Aplicado a	RF-26
Restricción origen RF	"Los colores primario y secundario deben especificarse en formato hexadecimal válido de seis dígitos (ej. #3A7BD5). Si el valor ingresado no cumple este formato, el

	sistema rechazará la operación e informará el error al usuario." *(RF-26 — Restricciones)* Flujo alterno: "El usuario ingresa un valor que no cumple el patrón hexadecimal. HTTP 400: Los colores deben expresarse en formato hexadecimal de 6 dígitos (ej. #3A7BD5). Verifique el valor ingresado en [CAMPO_COLOR]." *(RF-26 — Flujo alterno: Código de color hexadecimal inválido)*
Justificación	Es un invariante de formato de datos. Un color inválido puede romper la interfaz en todos los usuarios del sistema — es más crítico que una validación de texto libre.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_identidad_visual_colores_validos()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.primary_color IS NOT NULL
       AND NEW.primary_color !~ '^#[0-9A-Fa-f]{6}$' THEN
        RAISE EXCEPTION 'INVALID_COLOR: El color primario "%" no tiene
formato hexadecimal válido (#RRGGBB de 6 dígitos).',
            NEW.primary_color
            USING ERRCODE = 'P0135';
    END IF;

    IF NEW.secondary_color IS NOT NULL
       AND NEW.secondary_color !~ '^#[0-9A-Fa-f]{6}$' THEN
        RAISE EXCEPTION 'INVALID_COLOR: El color secundario "%" no tiene
formato hexadecimal válido (#RRGGBB de 6 dígitos).',
            NEW.secondary_color
            USING ERRCODE = 'P0135';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_identidad_visual_colores_validos
BEFORE INSERT OR UPDATE ON modulo9.identidad_visuales
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_identidad_visual_colores_validos();
```

TRG-M9-30 — Validación de theme_mode en temas visuales

Descripción	Valida que theme_mode sea exactamente 1, 2 o 3 (Claro, Oscuro, Sistema). Cualquier otro valor se rechaza.
Aplicado a	RF-27
Restricción origen RF	"El sistema deberá permitir seleccionar entre diferentes temas visuales: Tema claro (Light mode), Tema oscuro (Dark mode), Tema automático según configuración del dispositivo." *(RF-27 — Descripción)* Flujo alterno: "El sistema recibe un theme_mode fuera del rango permitido (distinto a 1, 2 o 3). HTTP 400: El identificador de tema [VALOR_RECIBIDO] no es válido. Los valores permitidos son 1 (Claro), 2 (Oscuro) o 3 (Automático)." *(RF-27 — Flujo alterno: Valor de tema inválido)*
Justificación	Dominio cerrado de valores. Es equivalente a un ENUM que el DDL no modela así.

```
CREATE OR REPLACE FUNCTION modulo9.trg_fn_tema_visual_valor_valido()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.theme_mode NOT IN (1, 2, 3) THEN
        RAISE EXCEPTION 'INVALID_THEME: El tema visual "%" no es válido.
Los valores permitidos son: 1 (Claro), 2 (Oscuro), 3
(Automático/Sistema).',
            NEW.theme_mode
            USING ERRCODE = 'P0136';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_tema_visual_valor_valido
BEFORE INSERT OR UPDATE OF theme_mode ON modulo9.temas_visuales
```

```
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_tema_visual_valor_valido();
```

TRG-M9-31 — Validación de configuración remota: lógica de tiempos

Descripción	Valida que frecuencia_captura > 0, intervalo_transmision > 0, y que intervalo_transmision >= frecuencia_captura. Misma lógica de coherencia que RF-18 pero a nivel de dispositivo individual.
Aplicado a	RF-23
Restricción origen RF	"Los parámetros de configuración deben cumplir reglas de validación, incluyendo: valores enteros positivos para frecuencia de captura e intervalo de transmisión." *(RF-23 — Restricciones)* > Flujo alterno: "El usuario ingresa una frecuencia de captura o intervalo de transmisión que excede los límites definidos. HTTP 400: El parámetro [NOMBRE_PARAMETRO] debe estar entre [MIN] y [MAX] minutos." *(RF-23 — Flujo alterno: Parámetros fuera de rango técnico)* > Flujo alterno: "El usuario intenta configurar un intervalo_transmision menor a la frecuencia_captura. HTTP 400: El intervalo de transmisión no puede ser menor a la frecuencia de captura de datos." *(RF-23 — Flujo alterno: Inconsistencia lógica de tiempos)*
Justificación	Invariante matemático de coherencia operativa. Un dispositivo no puede transmitir más rápido de lo que captura datos.

```
CREATE OR REPLACE FUNCTION
```

```
modulo9.trg_fn_configuracion_remota_tiempos_validos()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.frecuencia_captura IS NULL OR NEW.frecuencia_captura <= 0
THEN
        RAISE EXCEPTION 'INVALID_CONFIG: La frecuencia de captura debe
ser un entero positivo mayor a 0. Valor recibido: %.',
            NEW.frecuencia_captura
            USING ERRCODE = 'P0137';
    END IF;

    IF NEW.intervalo_transmision IS NULL OR NEW.intervalo_transmision <=
0 THEN
        RAISE EXCEPTION 'INVALID_CONFIG: El intervalo de transmisión
debe ser un entero positivo mayor a 0. Valor recibido: %.',
            NEW.intervalo_transmision
            USING ERRCODE = 'P0137';
    END IF;

    IF NEW.intervalo_transmision < NEW.frecuencia_captura THEN
        RAISE EXCEPTION 'INVALID_CONFIG: El intervalo de transmisión (%)
no puede ser menor a la frecuencia de captura (%). No se puede
transmitir antes de capturar.',
            NEW.intervalo_transmision, NEW.frecuencia_captura
            USING ERRCODE = 'P0138';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_configuracion_remota_tiempos_validos
BEFORE INSERT OR UPDATE ON modulo9.configuraciones_remotas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_configuracion_remota_tiempos_validos();
```

TRG-M9-32 — Unicidad de configuración remota PENDIENTE por dispositivo

Descripción	Impide crear una nueva configuración remota si ya existe una en estado PENDIENTE para el mismo dispositivo. Implementa el flujo alterno "Conflicto de comandos concurrentes" del RF-23.
Aplicado a	RF-23
Restricción origen RF	Flujo alterno: "Un administrador intenta enviar una configuración mientras existe otra en estado PENDIENTE para el mismo dispositivo. HTTP 409: Ya existe una configuración pendiente de aplicación para el dispositivo [SERIAL]. Espere a que se aplique o cancele la anterior antes de enviar una nueva." *(RF-23 — Flujo alterno: Conflicto de comandos concurrentes)*
Justificación	Invariante de integridad operativa. Dos comandos PENDIENTE para el mismo dispositivo son incoherentes y podrían causar comportamiento impredecible en el hardware.

```
CREATE OR REPLACE FUNCTION
modulo9.trg_fn_configuracion_remota_sin_pendiente()
RETURNS TRIGGER AS $$
DECLARE
    v_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM modulo9.configuraciones_remotas
    WHERE id_dispositivo_iot = NEW.id_dispositivo_iot
      AND estado             = 'PENDIENTE';

    IF v_count > 0 THEN
        RAISE EXCEPTION 'PENDING_CONFIG_EXISTS: Ya existe una
configuración PENDIENTE para el dispositivo ID %. Espere a que se
aplique o cancélela antes de enviar una nueva.',
            NEW.id_dispositivo_iot
            USING ERRCODE = 'P0139';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_configuracion_remota_sin_pendiente
BEFORE INSERT ON modulo9.configuraciones_remotas
FOR EACH ROW
EXECUTE FUNCTION modulo9.trg_fn_configuracion_remota_sin_pendiente();
```